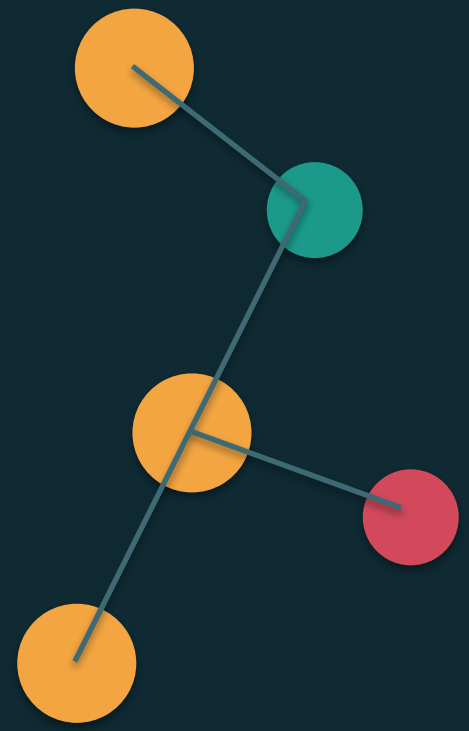
# The spiking neuron, in hardware

What a neuron is, what LIF means, how it is built in logic, and why it fits a line-rate packet classifier



SNN-based network packet classifier · review

# What this review covers

1

## A neuron

biological idea → artificial neuron → the spiking neuron

2

## LIF

leak, integrate, fire; the model our chip uses

3

## In hardware

one neuron as a logic diagram; the real RTL

4

## Vs other models

IF, ALIF, TTFS, Izhikevich — and why LIF fits

5

## Numbers

power & area, compared with the literature

# What is a neuron?

*The basic decision unit of every neural network*

## Biological

- Dendrites collect electrical charge from other neurons
- The cell body adds it up
- When the charge passes a threshold, the neuron fires a short pulse (a spike) down its axon
- Then it resets and starts again

## Artificial (what we build)

$$\text{output} = f\left( \Sigma\, w_i \cdot x_i + b \right)$$

- **Inputs $x_i$** × weights $w_i$, summed
- **Weights** are what training learns
- **A function f** decides the output
- A standard (ANN) neuron outputs one number

# The spiking neuron: pulses, not numbers

## Standard neuron

- Outputs one number (e.g. 0.73)
- Inputs are numbers
- Needs multiplications
- No sense of time / no memory

## Spiking neuron (ours)

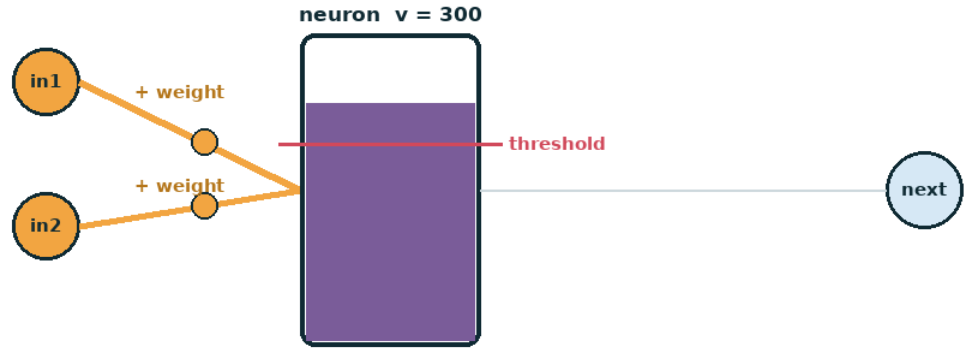
- Outputs spikes: 1 or 0 at each time step
- Inputs are spikes too
- Weight × spike = add the weight, or not
- Keeps a charge over time (memory)

**Key point:** because inputs are 1-bit spikes, a spiking neuron needs NO multipliers — only adders. That is why it is cheap in silicon.

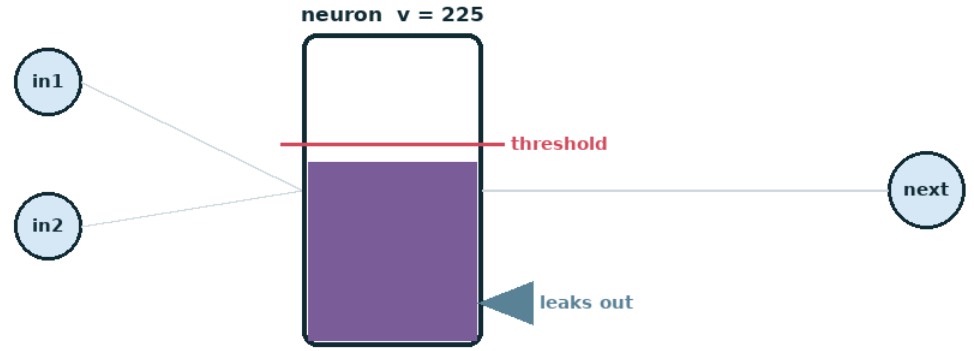
# LIF = Leak, Integrate, Fire

*The neuron model our chip uses, shown as a leaky bucket*

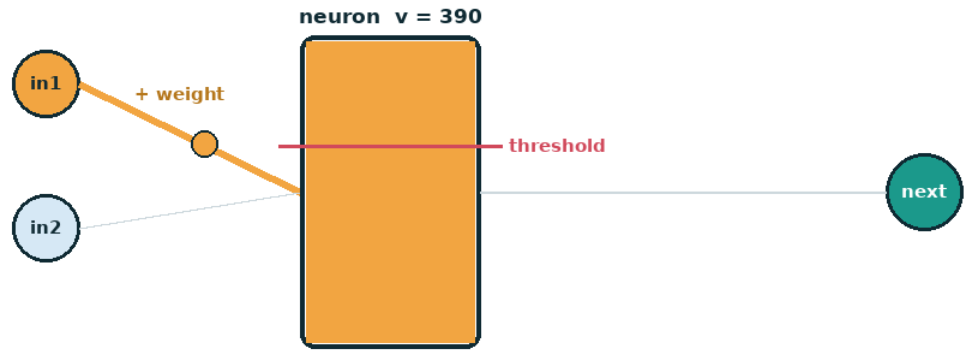
**Step 2: integrate**




**Step 3: leak**

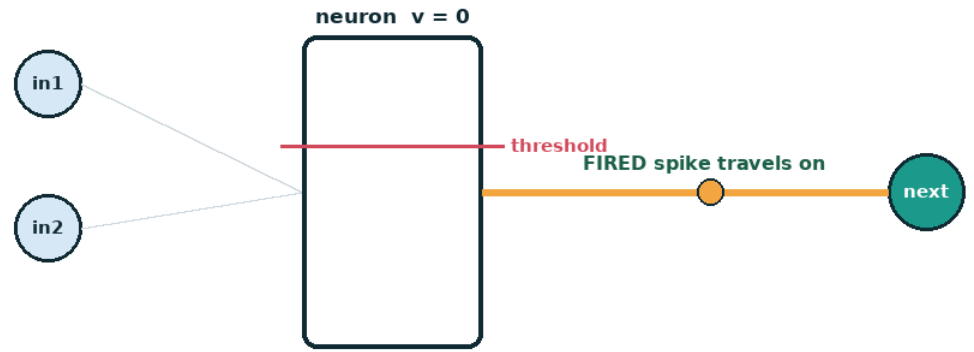



**Step 5: fire**




Charge crosses the threshold! The neuron fires a single 1-bit spike (and will reset to 0).

**Step 6: spike travels, then reset**

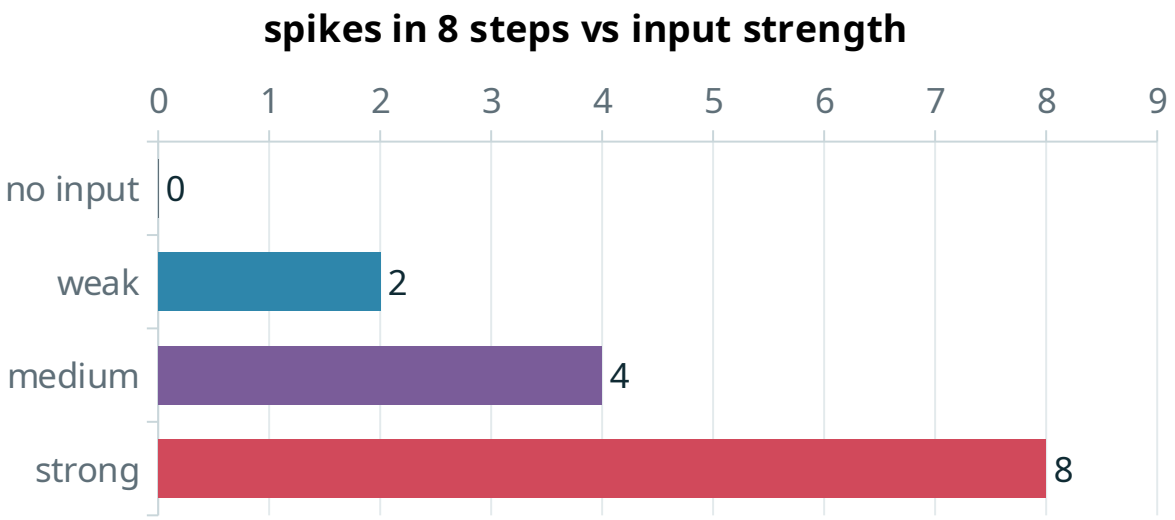



The FIRED spike is what reaches the next layer. The charge resets to 0, ready to start again.

# How a LIF neuron behaves

$$v \leftarrow v - v/4 + I$$
$$\text{if } v \geq \theta\text{: spike, then } v \leftarrow 0$$

*leak (÷4) · integrate (+I) · fire & reset, repeated 8 times*



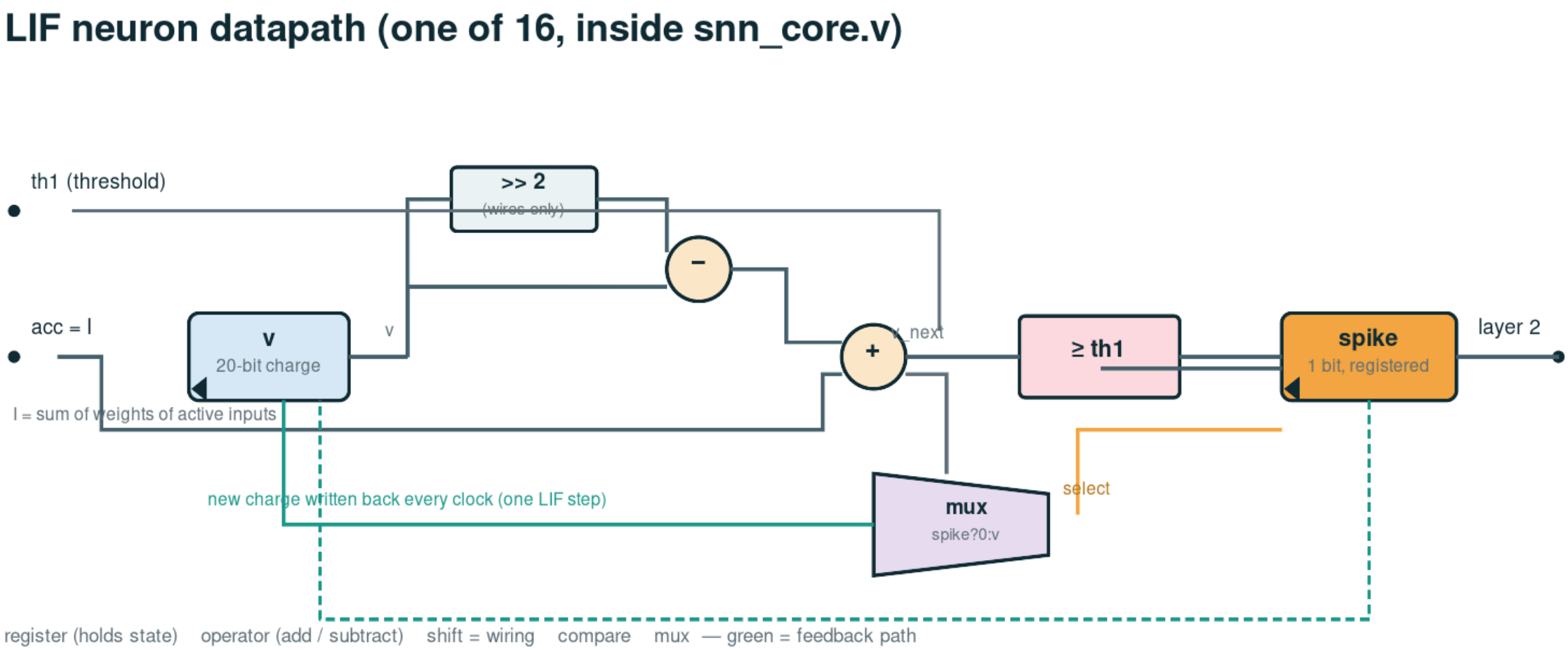

## What this means

**No / negative input** → 0 spikes (stays quiet)
**Weak input** → fires slowly (2 of 8)
**Strong input** → fires every step (8 of 8)

**The spike count encodes strength.**
The 'leak' makes it forget old input, so it reacts to RECENT activity — useful for spotting bursts and floods.

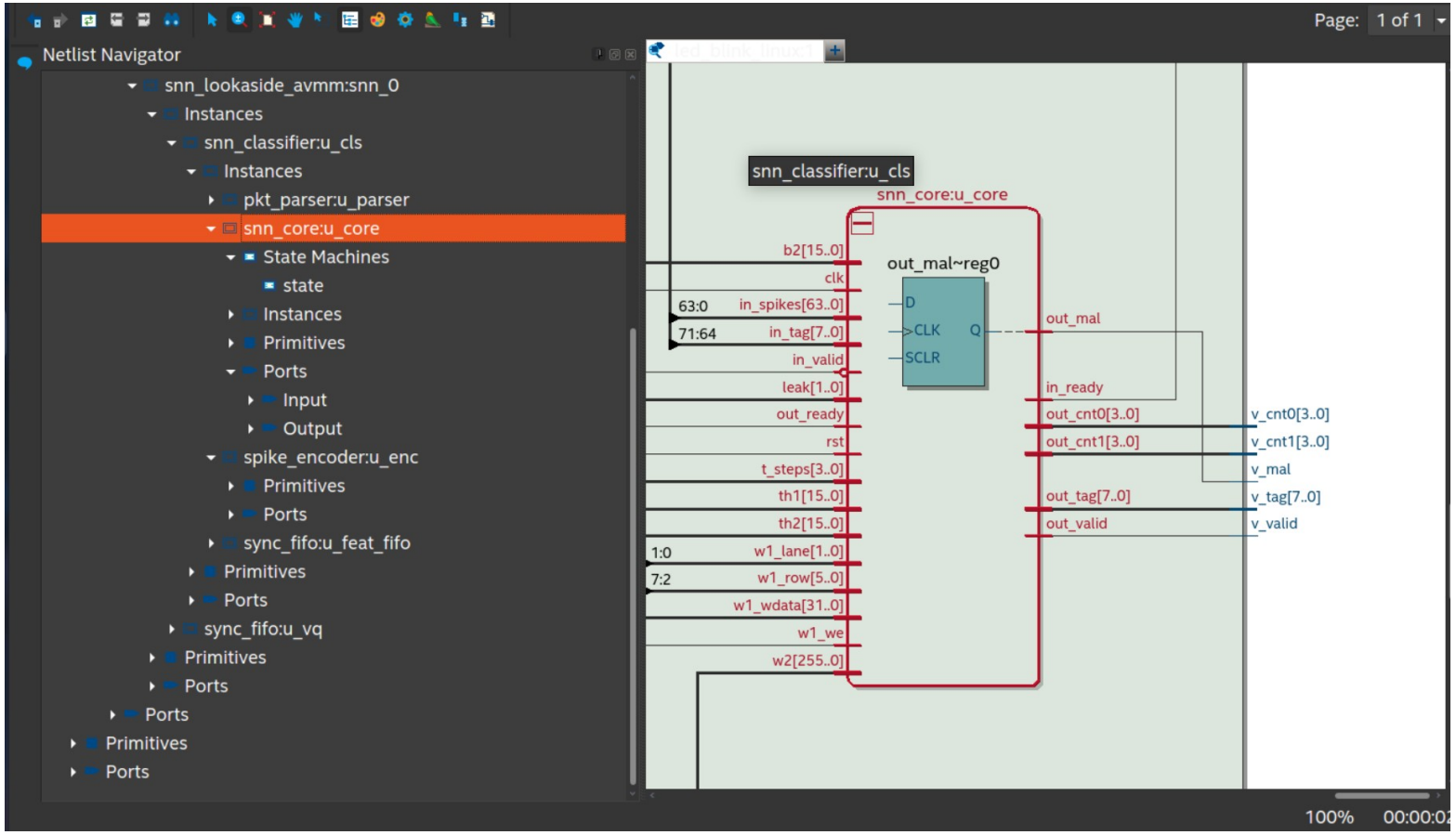
Hardware cost: 1 register + 2 adders + 1 comparator + 1 reset mux. No multiplier.

# One LIF neuron in hardware (logic diagram)




*register (charge) → >>2 & subtract (leak) → + I (integrate) → compare (fire) → reset mux · green = write-back each clock*

# The real hardware: snn_core in Quartus RTL Viewer




## What you see

**Left:** in_spikes[63:0] and the weights (w1, w2, b2), thresholds, leak

**Right:** out_cnt0/1 (spike counts), out_mal (verdict)

**out_mal~reg0:** the single flip-flop holding the verdict

Inside: the IDLE/ACC/LIF/OUT state machine + 16+2 LIF neurons

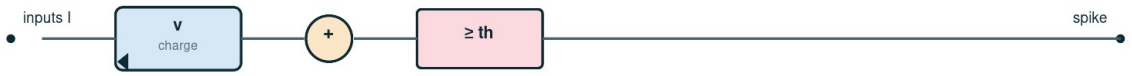
# LIF vs other neuron models

| model | how it works | hardware per neuron | trade-off |
|---|---|---|---|
| **IF (integrate-and-fire)** | like LIF but no leak | register, adder, compare | cheapest; never forgets, noisy |
| **LIF (ours)** | leak ¼ each step, fire, reset | register, 2 adders, compare, mux | good balance; standard choice |
| **Adaptive LIF (ALIF)** | threshold rises after a spike | +1 register, +1 adder | longer memory; for slow scans |
| **Time-to-first-spike** | fire once, timing = value | LIF + a timer | very low power [Zhou & Li, ICPR 2020] |
| **Izhikevich** | 2 variables, quadratic ($v^2$) | MULTIPLIER + 2 registers | biologically rich; several× larger |

*Diagrams of each on the next slide; the SNN-vs-CNN power question is studied by Plagwitz et al. [ACM TECS, 2023].*

# The models as logic: simplest → richest

### IF neuron (integrate-and-fire): the simplest



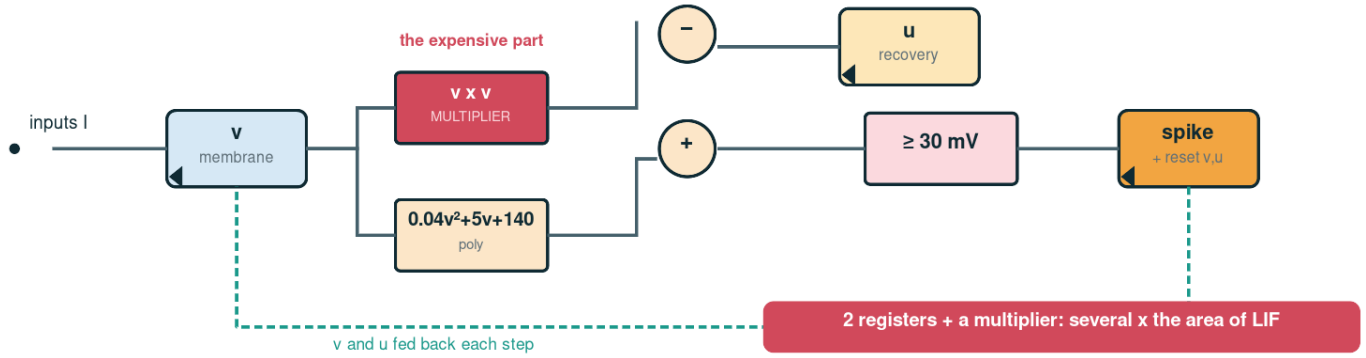

No leak: the charge only ever goes up until it fires. 1 register + 1 adder + 1 comparator.
Cheapest to build, but it never forgets, so noise can slowly push it over the line.

### LIF neuron (ours): leaky integrate-and-fire




Adds a leak (lose 1/4 each step) before integrating. register + 2 adders + compare + reset mux.
The standard choice: forgets old input, so it reacts to recent activity. This is what our chip uses.

### Izhikevich neuron: biologically rich, but needs a multiplier




## Why LIF for us

**No multiplier** (unlike Izhikevich)

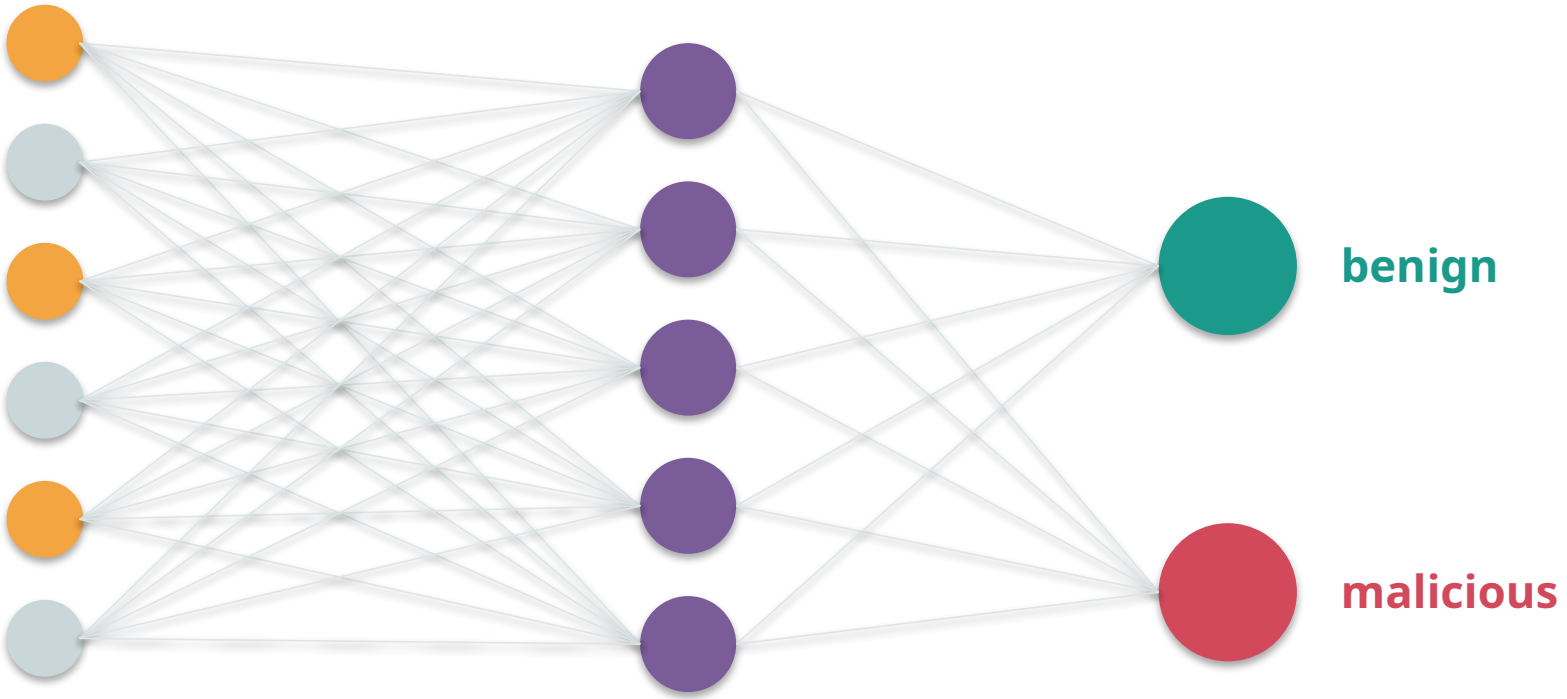
**Has memory** (unlike IF)

Small, fast, and the leak means it reacts to recent bursts — exactly what flood/scan detection needs.

Trained offline so the int8 weights run bit-exactly on the chip [Neftci et al., IEEE SPM 2019].

# From one neuron to a network (SNN)

*Our network: 64 → 16 → 2*




64 inputs (spikes)

16 hidden LIF

## The decision

Each of the 18 neurons is a LIF neuron.

Run 8 time steps.

**Winner = the output neuron that fired more.**

Weights (int8) are loaded at run time.

# Why an SNN fits our application

*A packet classifier inside the network card, at line rate*

### 1 No multipliers

spikes make every weight an add; 0 DSP blocks used on FPGA → small, low power

### 2 Event-driven

only the ~13 of 64 'active' header features are processed per packet

### 3 Memory over time

the leak + a rate window catch SYN floods and scans that single packets hide

### 4 Fast enough

~28 clock cycles per packet — keeps up with 1 Gb/s

### 5 Retrain, don't rebuild

weights are loaded at run time; no new chip needed

### 6 Neuromorphic path

on-chip learning (STDP) possible later

# Our hardware cost (power & area)

**1.28 mm²**

sky130 die, placed & routed

**125 mW**

at 100 MHz (sky130, typical)

**0**

multipliers / DSP blocks

**6 %**

of a Cyclone V FPGA

**~28**

clock cycles per packet

**28 k**

flip-flops (mostly buffer+weights)

*sky130 synthesis: SNN core 0.39 mm². Cadence 90 nm (Genus) area/timing/power: being finalised on the uni server.*

# Our approach vs the literature

| work | platform | multipliers | vs ours |
|---|---|---|---|
| **Alom & Taha [IJCNN 2017]** | TrueNorth chip | — | SNN IDS 90.1%, but off the data path (software feeds it) |
| **Zhou & Li [ICPR 2020]** | software / sim | — | temporal-coded SNN IDS; no line-rate hardware |
| **Taurus [ASPLOS 2022]** | switch ASIC | many | per-packet ML in-switch, but non-spiking (DNN/SVM) |
| **Plagwitz et al. [ACM TECS 2023]** | FPGA | SNN: none | shows SNN area/power can beat CNN only if sparse |
| **FPGA SNN accel. [VTC 2024]** | FPGA | none | image benchmarks; not fused with a NIC |
| **Ours** | ASIC/FPGA NIC | **0** | SNN fused in the NIC datapath, line rate, 0 DSP |

*Energy-efficient neuromorphic design: Na & Kim [Electronics 2025]; algorithm–hardware co-design: Nature Mach. Intell. 2026.*

# References

## Journal papers


**[J1]** Neftci, Mostafa, Zenke, “Surrogate Gradient Learning in SNNs,” IEEE Signal Processing Magazine, 2019.

**[J2]** Plagwitz, Hannig, Teich, Keszöcze, “To Spike or Not to Spike? SNN vs CNN on FPGA,” ACM Trans. Embedded Computing Systems, 2023.

**[J3]** Na, Kim, “An Energy-Efficient Neuromorphic Processor … for Edge AI,” Electronics (MDPI), 2025.

**[J4]** “Algorithm–hardware co-design of neuromorphic networks with dual memory pathways,” Nature Machine Intelligence, 2026.


## Conference papers


**[C1]** Alom, Taha, “Network Intrusion Detection … on Neuromorphic Computing System,” IEEE IJCNN, 2017.

**[C2]** Zhou, Li, “SNNs with Single-Spike Temporal-Coded Neurons for Network Intrusion Detection,” IEEE ICPR, 2020.

**[C3]** Swamy et al., “Taurus: A Data Plane Architecture for Per-Packet ML,” ACM ASPLOS, 2022.

**[C4]** “Efficient Hardware Acceleration of SNNs using FPGA,” IEEE VTC2024-Spring, 2024.




Datasets: Moustafa & Slay, UNSW-NB15 (MilCIS 2015); Sharafaldin et al., CIC-IDS2017 (ICISSP 2018).